MULTI-STEP PROMPT WORKFLOW

Mini Project: Student Attendance Management System

Name: P. Dhruvakumar

Registration Number: 99230040719

Date: 12-11-2026

Scenario: Transforming a mini-project idea into a structured project proposal

1. Project Overview

Project Title: Student Attendance Management System

The Student Attendance Management System is a simple web-based application designed to help teachers maintain student attendance records. It allows authorized users to add student details, record daily attendance, view attendance information, and search records. The project aims to reduce manual register work and keep attendance information organized.

Suggested Technology: HTML, CSS, JavaScript, Python, Flask, MySQL

Users: Teachers, students, and administrators

Core Features: Student entry, class/section details, daily attendance marking, attendance record search, attendance summary, and database storage.

2. Raw Project Notes

Many educational institutions record student attendance manually in notebooks or spreadsheets. This can take time and make it difficult to find past attendance, check a student's attendance status, and maintain organized records.

The proposed Student Attendance Management System should allow teachers to add student details, mark students as present or absent, view and search attendance records, and check simple attendance summaries. The system should store information in a database and provide an interface that is easy for beginners to use.

STEP 1 — ANALYSIS & PROJECT PLANNING

Exact Prompt Used

Act as an expert software project analyst and technical documentation writer. I want to develop a mini project called “Student Attendance Management System”. Analyze the raw project notes below and produce: (1) the main problem statement, (2) exactly 4 key project objectives, (3) the target users, (4) exactly 6 core functional requirements, (5) a simple technology stack using HTML, CSS, JavaScript, Python, Flask, and MySQL, and (6) a structured outline for a 700-word mini-project proposal. Do not invent advanced features that are not supported by the notes.

Output format: Problem Statement – Objectives – Target Users – Functional Requirements – Technology Stack – Proposal Outline

Raw notes: Many educational institutions record attendance manually in notebooks or spreadsheets. This takes time and makes it difficult to find past attendance, check student attendance status, and maintain organized records. The proposed system should allow teachers to add student details, mark students as present or absent, view and search attendance records, check simple attendance summaries, and store information in a database. The interface should be easy for beginners to use.

Intermediate AI Response — Live Execution Record

Problem Statement
Manual attendance management using registers or spreadsheets can be time-consuming. It can also make it difficult to search previous records, check student attendance status, and maintain accurate attendance information.

Objectives

- Organize student details and attendance information in one system.
- Simplify the daily process of marking students as present or absent.
- Make attendance records easier to search and review.
- Provide clear and simple attendance summaries.

Target Users
Teachers, students, and administrators.

Functional Requirements

- Add and maintain student details.
- Maintain basic class and section information.
- Record daily attendance as present or absent.
- View attendance records for students.
- Search attendance records using available student details.
- Store attendance information and display simple attendance summaries.

Technology Stack
HTML and CSS create and style the web pages. JavaScript supports basic interaction. Python with Flask handles application logic and web requests. MySQL stores student and attendance records.

Proposal Outline
Introduction → Problem Statement → Objectives → Proposed Solution → Key Features → Technology Stack → Expected Benefits → Conclusion.

STEP 2 — PROPOSAL GENERATION

Exact Prompt Used

Excellent. Using only the project analysis, objectives, requirements, technology stack, and proposal outline from your previous response, write a complete mini-project proposal for the Student Attendance Management System. Strict constraints: maximum 700 words; use a professional academic tone; include Introduction, Problem Statement, Objectives, Proposed Solution, Key Features, Technology Stack, Expected Benefits, and Conclusion; explain how the system works in simple terms; do not introduce features or technologies not included in Step 1; use short paragraphs of no more than 3 sentences each; include this workflow: Teacher → Web Interface → Python/Flask Application → MySQL Database → Attendance Records/Summary. Output only the first draft of the proposal.

Intermediate AI Response — Live Execution Record

Introduction
The Student Attendance Management System is a simple web-based application designed to organize student attendance activities. It helps teachers maintain student details, record daily attendance, and review attendance information.

Problem Statement
Maintaining attendance manually in registers or spreadsheets can require considerable time and effort. It may also be difficult to search previous records, check a student's attendance status, and prepare a clear attendance summary.

Objectives
The project has four main objectives: to organize student and attendance information, simplify daily attendance marking, make attendance records easier to search and review, and provide simple attendance summaries.

Proposed Solution
The proposed system provides a simple web interface where teachers can enter student details and mark daily attendance as present or absent. The Python and Flask application processes the information and stores it in a MySQL database so that attendance records can be viewed and searched.

Workflow
Teacher → Web Interface → Python/Flask Application → MySQL Database → Attendance Records/Summary

Key Features
The system supports student entry, basic class and section information, daily attendance marking, viewing attendance records, searching records, and displaying simple attendance summaries. These functions cover the main requirements for maintaining organized attendance information.

Technology Stack
HTML provides the structure of the web pages, while CSS is used for page design. JavaScript supports basic interaction. Python and Flask manage application logic and web requests, while MySQL stores student and attendance records.

Expected Benefits
The system can reduce manual attendance record-keeping and make attendance information more organized. It can help teachers review records, check student attendance status, and view simple summaries more easily.

Conclusion
The Student Attendance Management System provides a simple approach to managing student attendance through a web-based application. By using a clear interface with Python, Flask, and MySQL, the project focuses on making routine attendance activities easier to maintain.

STEP 3 — CRITIQUE & REFINEMENT

Exact Prompt Used

Act as a strict senior project reviewer. Critically evaluate the proposal you just wrote against the following checklist: 1. Is it 700 words or fewer? 2. Does it contain all required sections? 3. Are exactly 4 objectives represented? 4. Are the six core functional requirements from Step 1 accurately reflected? 5. Does it use only the approved technology stack: HTML, CSS, JavaScript, Python, Flask, and MySQL? 6. Is the workflow clear and technically consistent? 7. Are unsupported or exaggerated claims avoided? 8. Is the language suitable for an academic mini-project submission? 9. Are paragraphs limited to a maximum of 3 sentences? 10. Is the conclusion clear? First, provide a “Self-Critique” with a bullet-by-bullet evaluation of all 10 criteria. Identify any weaknesses and explain how they should be fixed. Then provide “Final Polished Mini-Project Proposal”. Correct every issue you identify while preserving the useful content from the previous draft. The final proposal must remain within 700 words and satisfy all 10 criteria. Do not skip the self-critique.

Intermediate AI Response — Live Execution Record

Self-Critique

- Word limit: The proposal is below 700 words. No correction is required.
- Required sections: All eight required sections are included.
- Exactly 4 objectives: Four distinct objectives are stated in the Objectives section.
- Six functional requirements: Student entry, class/section information, attendance marking, viewing, searching, and summary/database support are reflected. The final version should make all six functions clear.
- Approved technology stack: Only HTML, CSS, JavaScript, Python, Flask, and MySQL are included.
- Workflow: The workflow follows the movement of information from the teacher interface to the application and database.
- Unsupported claims: No advanced or unsupported features are promised.
- Academic language: The wording is suitable for a college mini-project; a few sentences can be made more direct.
- Paragraph length: Paragraphs are within the three-sentence limit.
- Conclusion: The conclusion summarizes the purpose and technologies clearly.

Final Polished Mini-Project Proposal

Introduction
The Student Attendance Management System is a simple web-based application designed to organize student attendance activities. It helps teachers maintain student details and record daily attendance in an organized manner.

Problem Statement
Manual attendance management using registers or spreadsheets can make routine record-keeping time-consuming. It can also make it difficult to search previous attendance, check student status, and maintain accurate records.

Objectives
The project has four objectives: to organize student and attendance information, simplify daily attendance marking, make attendance records easier to search and review, and provide clear attendance summaries.

FINAL POLISHED MINI-PROJECT PROPOSAL (CONTINUED)

Proposed Solution
The proposed system provides a simple web interface where teachers can add student details and mark daily attendance as present or absent. Python with Flask processes the entered information, while MySQL stores student and attendance records for later viewing and searching.

Workflow
Teacher → Web Interface → Python/Flask Application → MySQL Database → Attendance Records/Summary

Key Features
The system provides six core functional areas: student entry, class and section information, daily attendance marking, viewing attendance records, searching attendance records, and storing information to display simple attendance summaries. These functions support the basic requirements for maintaining organized attendance records.

Technology Stack
HTML is used to create the structure of the web pages, and CSS is used for basic styling. JavaScript supports basic interaction, Python and Flask manage application logic and web requests, and MySQL stores student and attendance information.

Expected Benefits
The system can reduce manual record-keeping work and provide a more organized way to manage attendance. It can help teachers review attendance information, check student status, and view simple summaries more easily.

Conclusion
The Student Attendance Management System offers a simple and practical approach to managing student attendance through a web-based application. The project uses a beginner-friendly technology stack and focuses on the core requirements needed for organized attendance management.

4. Chaining Dependency

Step 1 → Step 2 → Step 3

Step 1 transforms the raw Student Attendance Management System notes into a structured analysis containing the problem statement, exactly four objectives, target users, six functional requirements, technology stack, and proposal outline. Step 2 uses that analysis to create the first complete proposal. Step 3 uses the Step 2 proposal as its input, performs a ten-point critique, and produces a corrected final proposal.

5. Critique Gate

Step 3 is the required critique gate. The AI first evaluates the previous proposal against all ten criteria and then refines the proposal so that the final version remains within 700 words and follows the approved requirements.

6. Live Execution Evidence

Run all three prompts sequentially in the same assigned AI interface. Add screenshots showing Prompt 1 and its response, Prompt 2 and its response, and Prompt 3 with the self-critique and final response. If the platform provides a shareable chat link, include it in the submission.

Evidence to insert:

- Screenshot 1: Step 1 prompt and complete analysis response.
- Screenshot 2: Step 2 prompt and first proposal response.
- Screenshot 3: Step 3 prompt, ten-point self-critique, and final polished proposal.

7. Assessment Checklist Requirement — Evidence in this workflow

- 3+ chained prompts — Three sequential prompts are provided.
- Explicit dependency — Steps 2 and 3 explicitly use the outputs from the preceding steps.
- Critique gate — Step 3 contains a ten-point review and correction stage.
- Mini-project scenario — Student Attendance Management System is used as the project.
- Live testing — Actual outputs and screenshots must be inserted after running the prompts.

8. Submission Note

This document provides the project idea, three linked prompts, sample draft responses, and the required workflow structure. For final submission, run the prompts in your assigned AI interface and replace the sample “Live Execution Record” responses with the exact outputs obtained. Add genuine screenshots or the chat link as required by your assessment instructions.

Student Details

Name: P. Dhruvakumar
Registration Number: 99230040719
Project: Student Attendance Management System